TEMPORARY JOB HIRING SYSTEM

Complete Project Report

Muhammad Hassan: 546696

Dinnar Haider: 545749

Muhammad Ashhad: 556125

Language	Java (JDK 8 or higher)
GUI Framework	Java Swing
Data Storage	Plain-text files (.txt)
Architecture	Role-Based OOP with Inheritance
IDE Recommended	IntelliJ IDEA / Eclipse / VS Code
External Libraries	None required

Table of Contents

1. Project Overview and Objectives

The Temporary Job Hiring System is a fully functional desktop application built in Java using the Swing GUI framework. It simulates a real-world temporary employment portal where three distinct types of users — Employees (job seekers), Companies (employers), and an Administrator — each log in to a personalized dashboard and perform role-specific operations.

All data is stored permanently in plain text files (.txt), so the system works entirely offline without any database, server, or third-party library. This makes it ideal as a student project that demonstrates core Java concepts including Object-Oriented Programming (OOP), File I/O, GUI development, and event-driven programming.

System Objectives

Objective	Description
Employee Portal	Employees can register, log in, browse all posted jobs, search by keyword, apply for positions, and track their application statuses.
Company Portal	Companies can register, post new job listings, view and manage their postings, see who applied, and hire applicants.
Admin Panel	The system administrator can view every user, job posting, and application across the entire system.
Persistent Storage	All data survives between application sessions using file-based storage — no data is lost when the app closes.
Role-Based Access	Each user type sees only their own dashboard. Employees cannot post jobs; companies cannot apply for jobs.
Zero Dependencies	The project compiles and runs with just the Java JDK — no Maven, Gradle, or external libraries required.

2. System Architecture and Design Philosophy

The project follows a layered architecture pattern, meaning each part of the code has a single, clear responsibility. This separation of concerns makes the code easier to understand, test, and maintain.

Layer	Classes / Files	Responsibility
Presentation Layer (GUI)	LoginScreen, RegisterScreen, AdminDashboard, CompanyDashboard, EmployeeDashboard, UITheme	Handles everything the user sees and interacts with. Built with Java Swing.
Business Logic Layer (Core)	User, Admin, Company, Employee, JobPosting, Application	Represents the real-world entities in the system. Contains the data and rules.
Data Access Layer (File I/O)	FileManager	The only class that reads from or writes to disk. All other classes call its methods.
Data Storage (Files)	users.txt, jobs.txt, applications.txt	Plain text files that store all persistent data between application sessions.

Design Principle: FileManager is the ONLY class that touches the .txt files. All dashboards call FileManager methods to load or save data. This means if you ever wanted to replace file storage with a database, you would only need to change FileManager — not every single dashboard.

3. Complete File Structure

The project consists of 15 Java source files and 3 data files. All .java files must be in the same folder to compile correctly.

File	Category	Purpose
Main.java	Entry Point	Starts the application; applies global UI theme; launches LoginScreen
User.java	Model (OOP)	Abstract base class; defines username, password, role fields
Admin.java	Model (OOP)	Admin subclass; role = ADMIN; toFileString saves ADMIN,u,p
Company.java	Model (OOP)	Company subclass; adds companyName field; role = COMPANY
Employee.java	Model (OOP)	Employee subclass; role = EMPLOYEE; simplest subclass
JobPosting.java	Model (Data)	Data class for one job listing; holds ID, title, description, company info
Application.java	Model (Data)	Data class for one job application; tracks status (Pending/Accepted)
FileManager.java	Data Layer	All file read/write logic; 13 public methods; no GUI code
UITheme.java	GUI Utility	Color palette, fonts, reusable components (buttons, fields, renderer)
LoginScreen.java	GUI Screen	Login form; validates credentials; routes to correct dashboard
RegisterScreen.java	GUI Screen	Registration form; dynamic company name field; saves new users
AdminDashboard.java	GUI Screen	Admin view; shows all users, jobs, applications in scrollable lists
CompanyDashboard.java	GUI Screen	Company view; post/delete jobs; view and hire applicants
EmployeeDashboard.java	GUI Screen	Employee view; real-time search; apply for jobs; track applications
users.txt	Data File	Comma-separated user records (ROLE,username,password[,companyName])
jobs.txt	Data File	Pipe-separated job records (id|title|desc|compUser|compName)
applications.txt	Data File	Pipe-separated application records including status field

4. Class Hierarchy and OOP Design

The project uses Object-Oriented Programming (OOP) principles throughout. The most important OOP concept here is inheritance — the User class is the parent, and Admin, Company, and Employee are its children.

Class Inheritance Tree

```
User (abstract class) -- cannot be instantiated directly
|
|--- Admin -- role: ADMIN
|    file format: ADMIN,username,password
|
|--- Company -- role: COMPANY
|    file format: COMPANY,username,password,companyName
|    extra field: companyName
|
|--- Employee -- role: EMPLOYEE
     file format: EMPLOYEE,username,password
```

Data Model Classes (Non-User)

In addition to the User hierarchy, the system has two data model classes that are not related by inheritance:

Class	Purpose	Key Fields
JobPosting	Represents one job listing posted by a company	jobId, title, description, companyUsername, companyName
Application	Represents one employee's application for a job	applicationId, employeeUsername, jobId, jobTitle, companyName, status

5. Detailed Class Explanations

5.1 User.java — Abstract Base Class

This is the foundation of the entire user system. It stores the three fields that every user type has in common: username, password, and role. By putting these in the parent class, we avoid repeating code in Admin, Company, and Employee.

```
public abstract class User {
    private String username; // login name
    private String password; // plain-text password
    private String role;     // 'ADMIN', 'COMPANY', or 'EMPLOYEE'

    public User(String username, String password, String role) {
        this.username = username;
        this.password = password;
        this.role = role;
    }

    // Getters
    public String getUsername() { return username; }
    public String getPassword() { return password; }
    public String getRole() { return role; }

    // Abstract: every subclass MUST provide this
    public abstract String toFileString();
}
```

Why abstract? You never need a plain 'User' object. You always need a specific type. Making the class abstract prevents accidental creation of incomplete User objects and forces each subclass to implement toFileString() in its own way.

5.2 Admin.java — Administrator Subclass

The simplest subclass. Admin adds no new fields — it just passes the role string 'ADMIN' to the parent User constructor and implements toFileString().

```
public class Admin extends User {
    public Admin(String username, String password) {
        super(username, password, 'ADMIN'); // passes role to parent
    }

    @Override
    public String toFileString() {
        return getRole() + ',' + getUsername() + ',' + getPassword();
        // Output: ADMIN,admin,admin123
    }
}
```

5.3 Company.java — Company/Employer Subclass

Company adds one extra field: companyName. This is the display name shown on all job postings (e.g., 'TechCorp Solutions'). The username is just the login ID (e.g., 'techcorp').

```
public class Company extends User {
    private String companyName; // e.g., 'TechCorp Solutions'

    public Company(String username, String password, String companyName) {
        super(username, password, 'COMPANY');
        this.companyName = companyName;
    }

    public String getCompanyName() { return companyName; }

    @Override
    public String toFileString() {
        return getRole()+','+getUsername()+','+getPassword()+','+companyName;
        // Output: COMPANY,techcorp,pass123,TechCorp Solutions
    }
}
```

5.4 Employee.java — Employee/Job Seeker Subclass

The simplest user type — like Admin, it adds no new fields. Its role string is 'EMPLOYEE'.

```
public class Employee extends User {
    public Employee(String username, String password) {
        super(username, password, 'EMPLOYEE');
    }

    @Override
    public String toFileString() {
        return getRole() + ',' + getUsername() + ',' + getPassword();
        // Output: EMPLOYEE,john,pass789
    }
}
```

5.5 JobPosting.java — Job Listing Data Model

A POJO (Plain Old Java Object) that stores all information about one job. It has no complex logic — just fields, a constructor, getters, and two string-formatting methods.

Field	Description
jobId	Unique identifier, e.g. JOB1001 or JOB1716000000000 (timestamp-based)
title	Job title, e.g. 'Software Developer'
description	Short job description, e.g. 'Develop web apps using Java'
companyUsername	Login username of the company who posted this job (used for filtering)
companyName	Display name of the company, e.g. 'TechCorp Solutions'

The toFileString() method formats the job for saving to jobs.txt:

```
public String toFileString() {
    return jobId + '|' + title + '|' + description + '|'
           + companyUsername + '|' + companyName;
    // Saved as: JOB1001|Software Developer|Develop web apps|techcorp|TechCorp
Solutions
}
```

Why pipes (|) instead of commas? Job descriptions can contain commas (e.g., 'Design, develop, and test software'). Using pipes as delimiters avoids confusion when splitting the line back into fields.

5.6 Application.java — Job Application Data Model

Tracks one employee's application for one job. The most important field is status, which starts as 'Pending' and can be changed to 'Accepted' when the company hires the applicant.

Field	Description
applicationId	Unique ID, e.g. APP2001 or APP+timestamp
employeeUsername	Who applied — the employee's login username
jobId	Which job they applied for
jobTitle	Stored for display even if the job is later deleted
companyName	Stored for display even if the job is later deleted
status	Current state: 'Pending', 'Accepted', or 'Rejected'

Why store jobTitle and companyName inside Application? This is called denormalization. If a company deletes their job posting, the application record still shows the correct title and company name. Without this, the displayed info would become blank or broken after job deletion.

5.7 FileManager.java — The Data Access Layer

This is the most important utility class in the project. It handles every single read and write operation. All 13 methods are static, meaning you call them as FileManager.loadJobs() without creating a FileManager object.

Method Signature	What it Does
initFiles()	Creates missing .txt files; adds default admin if none exists
loadUsers()	Reads all lines of users.txt, parses each, returns List<User>
saveUser(User)	Appends one user record to the end of users.txt
userExists(String)	Checks if a username is already taken (prevents duplicates)
authenticate(String, String)	Finds a user matching both username AND password
loadJobs()	Reads jobs.txt line by line, returns List<JobPosting>
saveJob(JobPosting)	Appends one job record to the end of jobs.txt
deleteJob(String jobId)	Removes a job AND its applications (cascade delete)
loadApplications()	Reads applications.txt, handles 5-field and 6-field formats
saveApplication(Application)	Appends one application record to applications.txt
updateApplicationStatus(...)	Loads all apps, changes one status, rewrites file
deleteApplication(String)	Removes one application record, rewrites the file
generateId(String prefix)	Returns prefix + System.currentTimeMillis() as unique ID

The Delete-and-Rewrite Pattern Explained:

Text files cannot delete a single line in-place like a database can. To remove a record, FileManager uses this pattern:

```
// Step 1: Load everything into memory
List<JobPosting> jobs = loadJobs();

// Step 2: Remove the unwanted record
jobs.removeIf(j -> j.getJobId().equals(jobId));

// Step 3: Open file in OVERWRITE mode (false = not append)
PrintWriter pw = new PrintWriter(new FileWriter(JOBS_FILE, false));

// Step 4: Write all remaining records back
for (JobPosting j : jobs) pw.println(j.toFileString());
pw.close();
```

6. File Format Specification

All three data files are plain text. Each line = one record. Fields are separated by a delimiter character. Never edit these files while the app is running — your changes may be overwritten.

6.1 users.txt — Comma Separated Values

Format depends on role:

```
ADMIN,username,password
COMPANY,username,password,companyName
EMPLOYEE,username,password
```

Real examples from the project:

```
ADMIN,admin,admin123
COMPANY,techcorp,pass123,TechCorp Solutions
COMPANY,buildco,pass456,BuildCo Engineering
EMPLOYEE,john,pass789
EMPLOYEE,sarah,pass101
```

Parsing logic in loadUsers(): Each line is split by comma into a parts[] array. parts[0] is the role, parts[1] is the username, parts[2] is the password. For COMPANY, parts[3] is the company name. A switch statement on parts[0] creates the correct User subclass.

6.2 jobs.txt — Pipe Separated Values

Format: jobId|title|description|companyUsername|companyName

Examples:

```
JOB1001|Software Developer|Develop web applications using Java|techcorp|TechCorp Solutions
JOB1002|Data Analyst|Analyze business data and create reports|techcorp|TechCorp Solutions
JOB1003|Civil Engineer|Design and oversee construction projects|buildco|BuildCo Engineering
```

6.3 applications.txt — Pipe Separated Values

Format: applicationId|employeeUsername|jobId|jobTitle|companyName|status

Status values: Pending, Accepted, Rejected

Examples:

```
APP2001|john|JOB1001|Software Developer|TechCorp Solutions|Pending
APP2005|emma|JOB1005|UI/UX Designer|DesignHub Creative|Pending
APP2016|zara|JOB1001|Software Developer|TechCorp Solutions|Accepted
```

Backward Compatibility: loadApplications() handles lines with BOTH 5 fields (old format without status) and 6 fields (new format with status). If only 5 fields are found, the application defaults to 'Pending' status. This prevents crashes if old data files are used with the new code.

7. Application Flow — Step by Step

7.1 Application Startup

```
// Main.java
public static void main(String[] args) {
    UITheme.applyGlobalDefaults();            // Step 1: Set colors/fonts globally
    SwingUtilities.invokeLater(() ->           // Step 2: Schedule GUI on Event
Thread
        new LoginScreen().setVisible(true)    // Step 3: Show Login window
    );
}
```

Why invokeLater()? Java Swing is not thread-safe. All GUI operations must run on a special thread called the Event Dispatch Thread (EDT). SwingUtilities.invokeLater() schedules the GUI creation to happen on the EDT, preventing visual glitches and crashes.

7.2 Login Flow

Step	Description
Step 1	User types username and password in LoginScreen
Step 2	handleLogin() checks for empty fields — shows error if blank
Step 3	FileManager.authenticate(username, password) is called
Step 4	authenticate() loads all users from users.txt using loadUsers()
Step 5	Java Streams filter the list for a user with matching username AND password
Step 6	If no match: returns null — error dialog 'Invalid credentials!'
Step 7	If match found: returns the User object (could be Admin, Company, or Employee)
Step 8	openDashboard(user) is called — checks user.getRole() with a switch statement
Step 9	The correct dashboard window opens; LoginScreen is disposed

```
// openDashboard() — type casting to correct subclass
private void openDashboard(User user) {
    switch (user.getRole()) {
        case 'EMPLOYEE': new EmployeeDashboard((Employee) user).setVisible(true);
break;
        case 'COMPANY':  new CompanyDashboard((Company) user).setVisible(true);
break;
        case 'ADMIN':    new AdminDashboard((Admin) user).setVisible(true); break;
    }
}
```

7.3 Employee: Applying for a Job

1 EmployeeDashboard loads — calls loadJobs() to fill the JList

2 checkHiringStatus() runs — congratulates any newly-hired employee and clears the notification

3 Employee types in the search bar — DocumentListener fires filterJobs() on every keystroke

4 filterJobs() rebuilds the JList showing only jobs matching the query

5 Employee selects a job and clicks 'Apply for Selected'

6 applyForJob() checks if they already applied (prevents duplicates)

7 A new Application object is created and saved via FileManager.saveApplication()

8 Success dialog shown; application appears in 'My Applications'

7.4 Company: Posting a Job and Hiring

1 CompanyDashboard loads — loadJobs() filtered to show only THIS company's jobs

2 Company clicks 'Post New Job' — a JOptionPane dialog opens with Title and Description fields

3 On OK, validation checks for empty fields

4 A new JobPosting is created using FileManager.generateId('JOB') for the ID

5 FileManager.saveJob() appends the new job to jobs.txt

6 Company selects a posted job and clicks 'View Applicants'

7 A JDialog opens showing all Application records for that jobId

8 Company selects an applicant and clicks 'Hire / Accept'

9 FileManager.updateApplicationStatus() changes status to 'Accepted' and rewrites the file

10 Next time that employee logs in, checkHiringStatus() finds their Accepted application

8. GUI Architecture and Screen Guide

8.1 UITheme.java — The Design System

Instead of hardcoding colors and fonts in every screen, all visual styles live in UITheme.java. This keeps the appearance consistent across all screens and makes changes easy — update one constant and every screen updates.

Color Constants:

Constant	Color Value	Used For
PRIMARY	#6366f1 (Indigo)	Main action buttons, selected item highlight
ACCENT	#10b981 (Green)	Secondary buttons, company names in job list
DANGER	#ef4444 (Red)	Delete and Logout buttons
BG_DARK	#0f172a	Main window background (very dark navy)
BG_PANEL	#1e293b	Panels, button bars, form backgrounds
BG_CARD	#334155	Selected list items, cards
TEXT_PRIMARY	#f1f5f9	All main text — light for dark background
TEXT_SECONDARY	#94a3b8	Labels, subtitles, secondary info
BORDER_COLOR	#475569	Input field borders, list item dividers

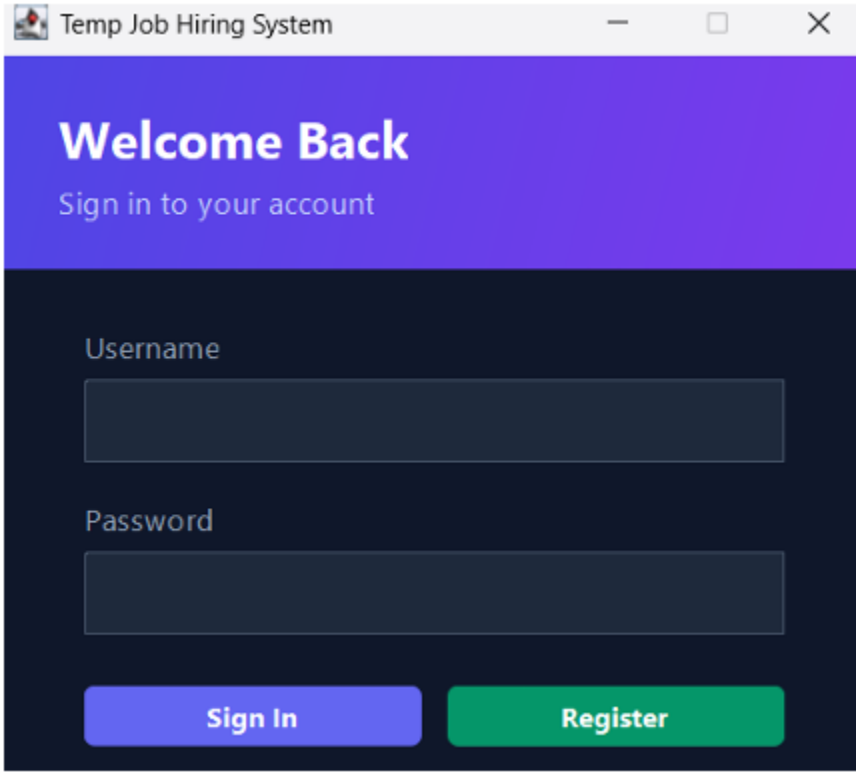
Factory Methods — Reusable Components:

Method	Returns	What It Creates
createPrimaryButton(text)	JButton	Indigo rounded button with hover darkening effect
createAccentButton(text)	JButton	Green rounded button with hover effect
createDangerButton(text)	JButton	Red rounded button for destructive actions
createTextField(cols)	JTextField	Dark-themed input with rounded border
createPasswordField(cols)	JPasswordField	Dark-themed password input (shows dots)
createSearchField(placeholder)	JTextField	Input with ghost placeholder text drawn when empty
createHeaderPanel(title, sub)	JPanel	Purple gradient header with title and subtitle labels
createLabel(text)	JLabel	Label with secondary text color applied

8.2 Screen-by-Screen Breakdown

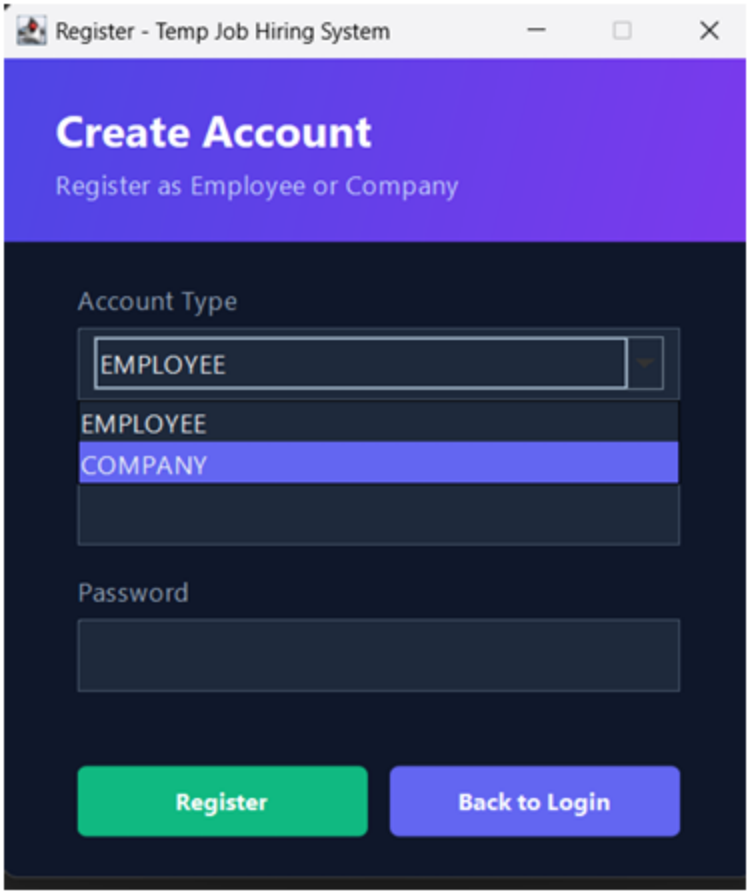
LoginScreen (420 x 380 px)

- Layout: BorderLayout — NORTH: gradient header, CENTER: form panel
- Form panel uses BoxLayout (Y_AXIS) to stack labels and fields vertically
- Username: JTextField | Password: JPasswordField
- Two buttons in a GridLayout(1,2): Sign In and Register
- Sign In → handleLogin() → FileManager.authenticate() → openDashboard()
- Register → new RegisterScreen().setVisible(true); dispose() closes login
- First screen shown. FileManager.initFiles() called here to set up data files.



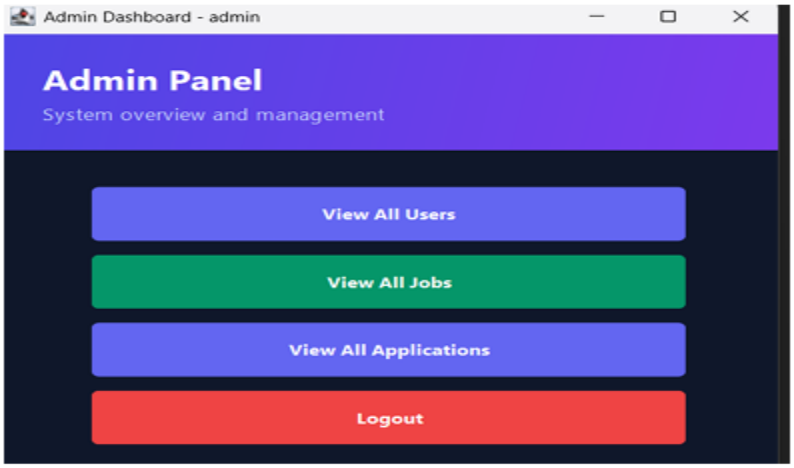
RegisterScreen (420 x 480 px)

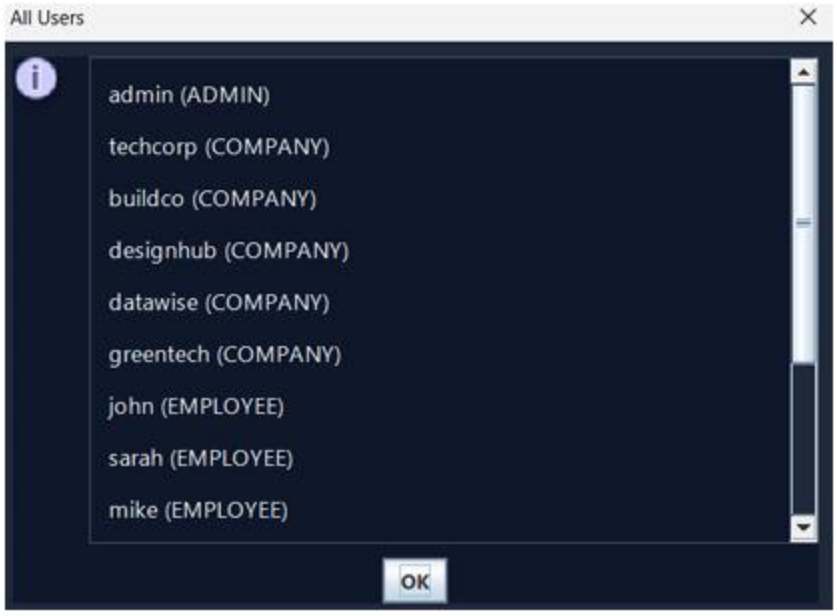
- Layout: BorderLayout — NORTH: header, CENTER: form
- JComboBox with options: EMPLOYEE, COMPANY
- ActionListener on combo dynamically shows/hides the Company Name field
- revalidate() + repaint() refresh the layout after show/hide
- Validation order: empty fields check → duplicate username check → save
- Company accounts require 4 fields; Employee accounts require 3

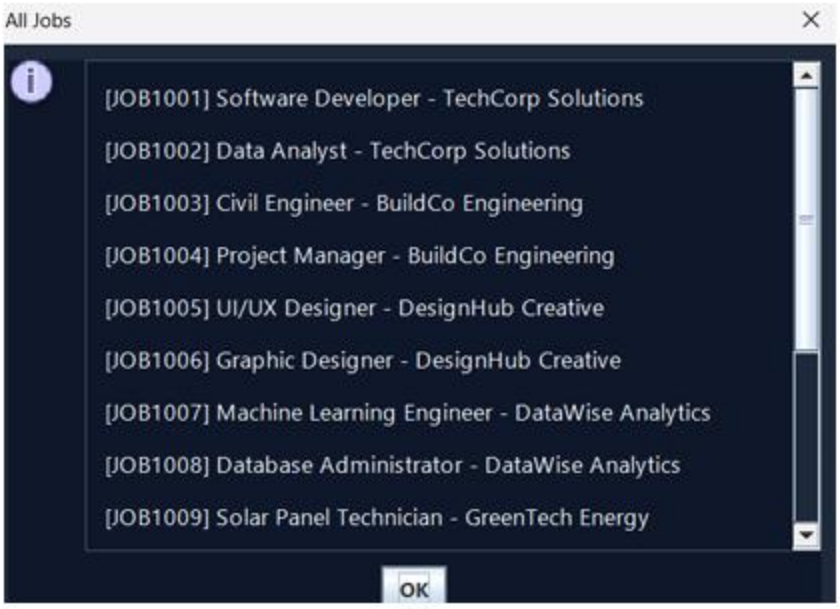


AdminDashboard (520 x 420 px)

- Layout: BorderLayout — NORTH: header, CENTER: card panel of buttons
- Center uses GridLayout(4,1) for four equal-height buttons
- showList() helper creates a JList in a JScrollPane inside JOptionPane
- Read-only: admin cannot create or delete anything from this dashboard
- Default credentials: admin / admin123

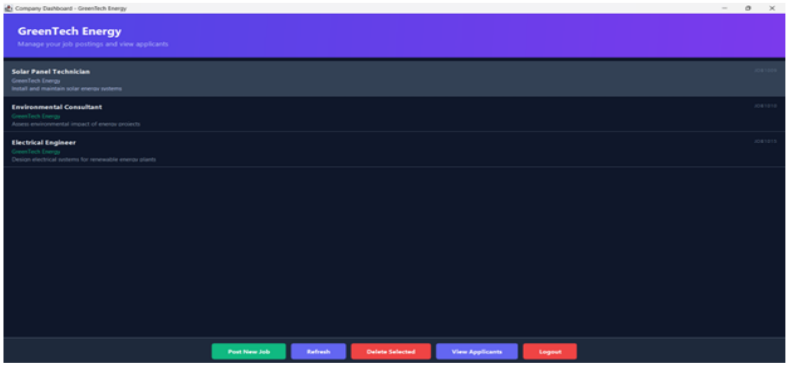


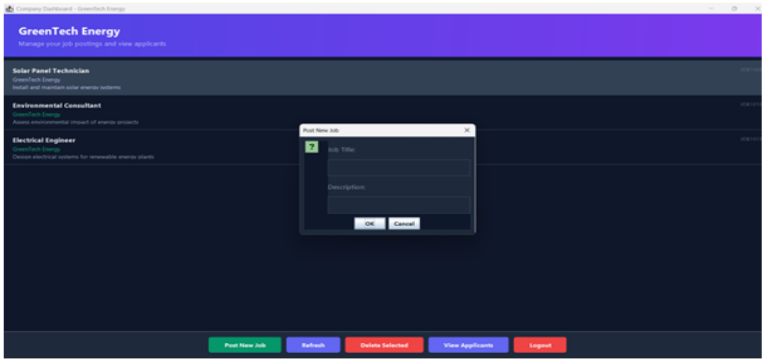

All Users
admin (ADMIN)
techcorp (COMPANY)
buildco (COMPANY)
designhub (COMPANY)
datawise (COMPANY)
greentech (COMPANY)
john (EMPLOYEE)
sarah (EMPLOYEE)
mike (EMPLOYEE)
OK

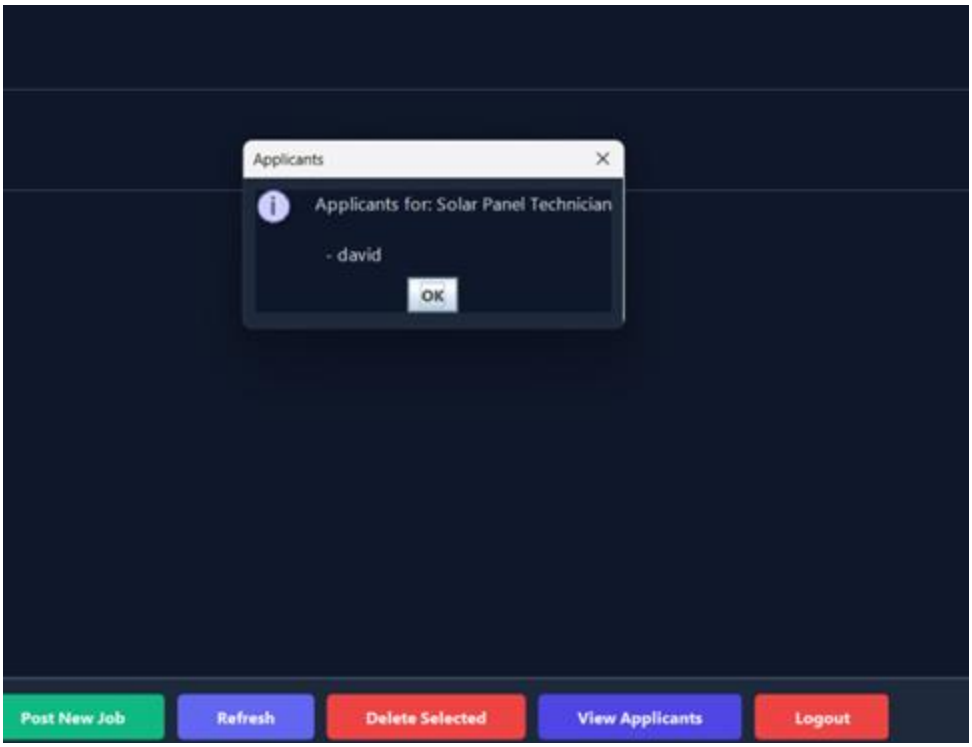

All Jobs
[JOB1001] Software Developer - TechCorp Solutions
[JOB1002] Data Analyst - TechCorp Solutions
[JOB1003] Civil Engineer - BuildCo Engineering
[JOB1004] Project Manager - BuildCo Engineering
[JOB1005] UI/UX Designer - DesignHub Creative
[JOB1006] Graphic Designer - DesignHub Creative
[JOB1007] Machine Learning Engineer - DataWise Analytics
[JOB1008] Database Administrator - DataWise Analytics
[JOB1009] Solar Panel Technician - GreenTech Energy
OK

CompanyDashboard (700 x 520 px)

- Layout: BorderLayout — NORTH: header, CENTER: job JList, SOUTH: button bar
- JList uses DefaultListModel and custom JobCellRenderer
- Jobs filtered to show only this company's postings using Stream.filter()
- Post Job: JOptionPane with two text fields; creates JobPosting with timestamp ID
- Delete: confirms via JOptionPane, calls deleteJob() which cascades to applications
- View Applicants: opens JDialog with Application list and Hire/Accept button
- Hire button calls updateApplicationStatus() to set status to 'Accepted'

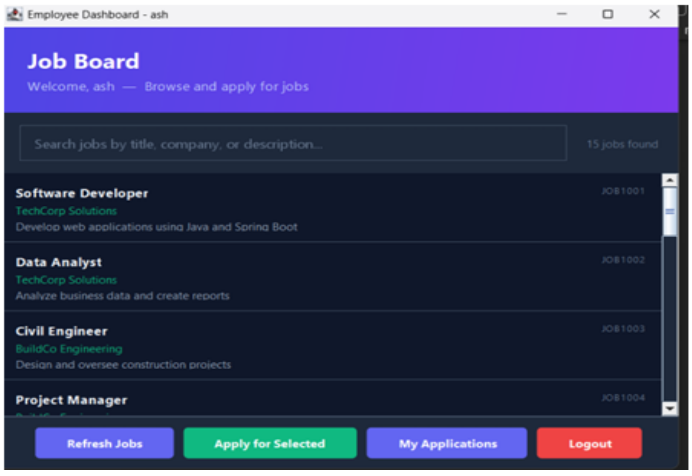


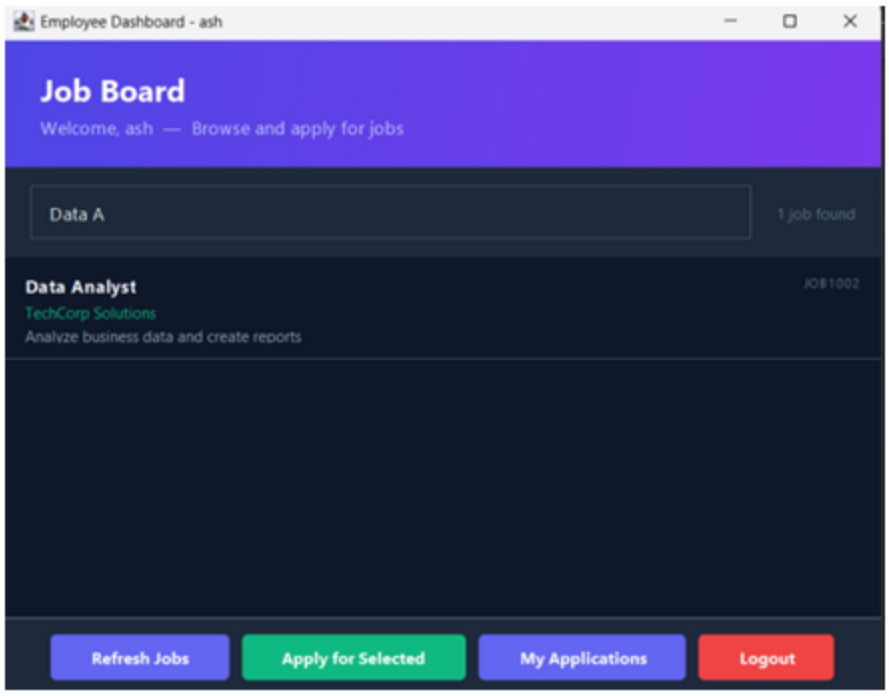


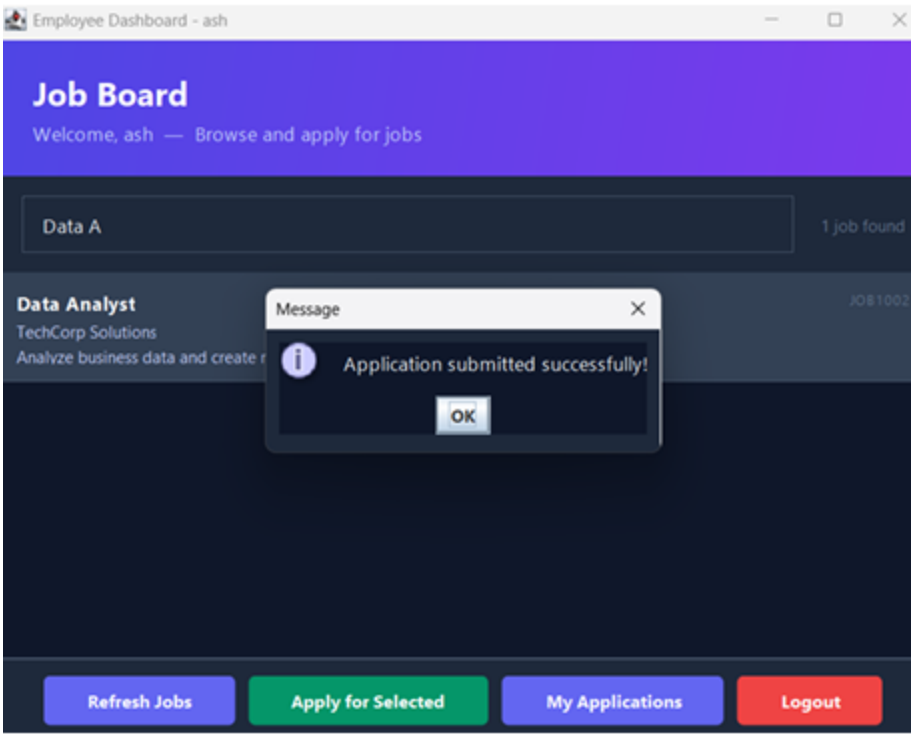


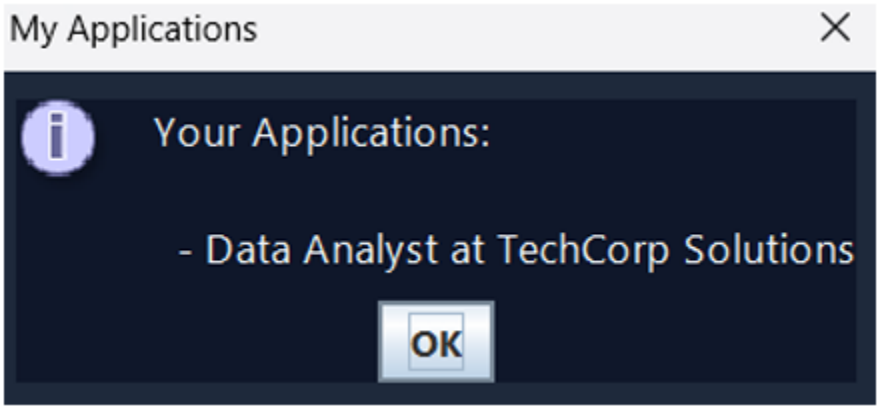
EmployeeDashboard (700 x 550 px)

- Layout: BorderLayout — NORTH: header, CENTER: search+list, SOUTH: button bar
- Search bar is a JPanel (BG_PANEL color) containing the search field and count label
- DocumentListener on search field triggers filterJobs() on every keystroke
- filterJobs() clears JList model and re-adds only matching jobs
- Real-time count label shows '5 jobs found' and updates instantly
- Apply button: checks for duplicate application, creates Application, calls saveApplication()
- My Applications: loads apps filtered by employeeUsername, shows in JOptionPane
- checkHiringStatus() on startup: finds Accepted apps, shows congratulations, deletes notification




Employee Dashboard - ash
Job Board
Welcome, ash — Browse and apply for jobs
Data A
1 job found
Data Analyst
JOB1002
TechCorp Solutions
Analyze business data and create reports
Refresh Jobs
Apply for Selected
My Applications
Logout


Employee Dashboard - ash
Job Board
Welcome, ash — Browse and apply for jobs
Data A
1 job found
Data Analyst
TechCorp Solutions
Message
Application submitted successfully!
OK
Refresh Jobs
Apply for Selected
My Applications
Logout


My Applications
Your Applications:
- Data Analyst at TechCorp Solutions
OK

8.3 JobCellRenderer — Custom List Items

The default JList only shows plain text from toString(). The JobCellRenderer makes each list item look like a proper job card with title, company, and description in different fonts and colors.

```
// JobCellRenderer extends JPanel implements ListCellRenderer<JobPosting>
// Structure of each list cell:
+-------------------------------------------------+----------+
| Job Title (bold, large)                         |          |
| Company Name (green, medium)                    | JOB1001  |
| Job description (gray, small)                   | (ID)     |
+-------------------------------------------------+----------+

// getListCellRendererComponent() is called by Swing for each visible item
// It updates colors based on isSelected parameter:
// selected = bright white text on BG_CARD background
// normal   = TEXT_PRIMARY text on BG_DARK background
```

8.4 Layout Managers Reference

Layout Manager	Used In	How It Works
BorderLayout	All JFrames and center panels	5 zones: NORTH, SOUTH, EAST, WEST, CENTER. Components stretch to fill their zone.
BoxLayout Y_AXIS	Login and Register form panels	Stacks components vertically. Used with Box.createVerticalStrut(n) for spacing.
GridLayout(r,c)	Admin button panel; Login button row	Divides space into equal grid cells. All cells same size.
FlowLayout	All dashboard button bars (SOUTH)	Places components left-to-right with gaps. Wraps to next row if needed.

9. Data Flow Diagrams (Text-Based)

9.1 Login Data Flow

```
USER ACTION          GUI LAYER        LOGIC LAYER          FILE LAYER
==========          =========        ===========          ==========
Type credentials -> LoginScreen
Click Sign In    -> handleLogin() -> FileManager
                                     .authenticate()   -> users.txt (read)
                 <- null (fail)    <- null
Show error dialog
                 <- User object    <- User object
                    openDashboard()
                 -> new Dashboard(user).setVisible(true)
```

9.2 Job Application Data Flow

```
EMPLOYEE ACTION      EMPLOYEE DASHBOARD    FILE MANAGER         FILES
===============      ==================    ============         =====
Dashboard opens   -> loadJobs()        -> loadJobs()        -> jobs.txt (read)
                  <- jobListModel filled<- List<JobPosting>  <-
Type in search    -> filterJobs() (in-memory filter, no file I/O)
Click Apply       -> applyForJob()
                     loadApplications() -> loadApplications()-> applications.txt
                     check duplicates   <- List<Application> <-
                     saveApplication()  -> saveApplication() -> applications.txt
(append)
Show success dialog
```

9.3 Hire Applicant Data Flow

```
COMPANY ACTION       COMPANY DASHBOARD    FILE MANAGER         FILES
==============       =================    ============         =====
Click View Apps   -> viewApplicants()
                     loadApplications() -> loadApplications()-> applications.txt
(read)
                     filter by jobId    <- List<Application> <-
Show JDialog
Click Hire/Accept -> updateStatus()
                  -> updateApplicationStatus()
                     load all apps      ->                    applications.txt
(read)
                     find + setStatus()
                     rewriteApplications()->                  applications.txt
(write)
Next employee login -> checkHiringStatus()
                     finds Accepted app -> loadApplications()-> applications.txt
                     shows congratulations dialog
                     deleteApplication()-> rewriteApplications()->applications.txt
```

9.4 Delete Job Data Flow (Cascade)

```
COMPANY ACTION       COMPANY DASHBOARD    FILE MANAGER         FILES
==============       =================    ============         =====
Click Delete      -> deleteJob()
                  -> deleteJob(jobId)
                     loadJobs()          ->                   jobs.txt (read)
                     removeIf matching
                     rewriteJobs()       ->                   jobs.txt (overwrite)
                     loadApplications()  ->                   applications.txt
(read)
                     removeIf matching jobId
                     rewriteApplications()->                  applications.txt
(overwrite)
                     loadJobs() refreshes UI
```

10. Code Verification and Quality Check

All source files were reviewed for correctness. The following table lists each verification check and its result:

Check	Result	Details
Compilation	PASS	All 15 .java files compile without errors. No missing imports.
Class References	PASS	All cross-class calls are valid. FileManager used correctly everywhere.
Inheritance Chain	PASS	Admin, Company, Employee all extend User; toFileString() implemented in each.
Abstract Contract	PASS	toFileString() is abstract in User; all subclasses provide implementation.
File I/O — Read	PASS	loadUsers(), loadJobs(), loadApplications() use try-with-resources correctly.
File I/O — Write	PASS	saveUser/saveJob/saveApplication use append mode (true). No data overwrite.
File I/O — Delete	PASS	deleteJob/deleteApplication use overwrite mode (false) with full rewrite.
Cascade Delete	PASS	deleteJob() also removes all applications for that jobId.
Duplicate Prevention	PASS	userExists() and alreadyApplied check prevent duplicate records.
Default Admin	PASS	initFiles() creates admin/admin123 if no ADMIN found in users.txt.
Null Handling	PASS	authenticate() returns null on failure; LoginScreen checks for null.
Empty List Guards	PASS	All listModel.isEmpty() checks prevent IndexOutOfBounds on empty lists.
GUI Thread Safety	PASS	SwingUtilities.invokeLater() used in Main.java for EDT compliance.
Backward Compat.	PASS	loadApplications() handles both 5-field and 6-field application records.
Status Lifecycle	PASS	Application status: Pending -> Accepted (via updateApplicationStatus).

11. Compilation and Execution Guide

11.1 Using Command Line (Terminal)

Step	Action
Step 1	Place ALL .java files in one folder, e.g. C:\TempJobSystem\
Step 2	Open Command Prompt (Windows) or Terminal (Mac/Linux)
Step 3	Navigate to the project folder: cd C:\TempJobSystem
Step 4	Compile all files at once
Step 5	Run the application
Step 6	The Login window will appear automatically

```
# Step 4 - Compile all Java files in the folder:
javac *.java

# Step 5 - Run the Main class:
java Main

# If you have spaces in path on Windows:
cd "C:\My Projects\TempJobSystem"
javac *.java
java Main
```

11.2 First Run Behavior

On the very first run: FileManager.initFiles() automatically creates users.txt, jobs.txt, and applications.txt in the same folder as the compiled .class files. It also creates the default admin account:

username: admin | password: admin123

You can change the admin password by editing users.txt directly while the app is closed.

12. Sample Data Reference

Company Accounts (from users.txt):

Login	Company Name / Notes
techcorp / pass123	TechCorp Solutions — posted: Software Dev, Data Analyst, Frontend Dev
buildco / pass456	BuildCo Engineering — posted: Civil Engineer, Project Manager, Site Supervisor
designhub / pass321	DesignHub Creative — posted: UI/UX Designer, Graphic Designer, Motion Designer
datawise / pass654	DataWise Analytics — posted: ML Engineer, Database Admin, BI Analyst
greentech / pass111	GreenTech Energy — posted: Solar Panel Tech, Environmental Consultant, Electrical Engineer

Employee Accounts (from users.txt):

Login	Application History
john / pass789	Applied for: Software Developer (JOB1001), Civil Engineer (JOB1003), ML Engineer (JOB1007)
sarah / pass101	Applied for: Software Developer (JOB1001), Environmental Consultant (JOB1010)
mike / pass202	Applied for: Data Analyst (JOB1002), Motion Designer (JOB1013)
emma / pass303	Applied for: UI/UX Designer (JOB1005), BI Analyst (JOB1014)
ali / pass404	Applied for: ML Engineer (JOB1007), Project Manager (JOB1004), Motion Designer
fatima / pass505	Applied for: Frontend Developer (JOB1011), Database Administrator (JOB1008)
david / pass606	Applied for: Solar Panel Technician (JOB1009), Electrical Engineer (JOB1015)
zara / pass707	Applied for: Graphic Designer (JOB1006), Software Developer (JOB1001)
ash / 111	Applied for: Data Analyst (JOB1002)

Admin Account:

Username: admin | Password: admin123 | Created automatically on first run if no ADMIN exists in users.txt.

13. Error Messages and Troubleshooting

Error Message	Cause	Solution
'Invalid credentials!'	Username or password does not match any user in users.txt	Re-check spelling. Default admin: admin / admin123
'Username already exists!'	Attempting to register with a username already in users.txt	Choose a different username
'Please fill all fields.'	A required input field was left empty	Complete all visible form fields before submitting
'Please select a job.'	Clicked Apply or Delete without selecting a job in the list	Click on a job in the list to highlight it first
'You already applied for this job!'	Employee tried to apply for a job they already applied for	Each job can only be applied to once per employee
'No applicants for this job.'	No employees have applied for the selected job yet	Wait for employees to discover and apply for the posting
'Please enter company name.'	Registering as COMPANY without filling the Company Name field	Fill in the Company Name field that appears for COMPANY accounts
'This applicant is already hired.'	Company tried to hire someone already marked Accepted	The applicant is already hired — no action needed

Common Runtime Issues:

Symptom	Likely Cause	Fix
App doesn't start / blank window	GUI code outside EDT	Ensure Main.java uses SwingUtilities.invokeLater()
Login always fails for new user	saveUser() using wrong mode	Confirm FileWriter(file, true) — true = append
All data lost on restart	saveUser using overwrite mode	Use FileWriter(file, true) for all save methods
Job list shows all companies' jobs	Missing filter in CompanyDashboard	Check stream().filter(j -> j.getCompanyUsername().equals(user))
ArrayIndexOutOfBoundsException	Malformed line in .txt file	Check for missing pipes or commas in data files
NullPointerException on login	authenticate() returned null not checked	Check if(user == null) before calling getRole()

14. Limitations and Future Improvements

Current Limitations

Plain-text passwords: Passwords are stored in plain text in users.txt. Anyone who opens the file can see all passwords. In a real system, passwords must be hashed (scrambled) before storing.

No input sanitization: If a username contains a comma, it could break the parsing of users.txt. The system does not check for or escape special delimiter characters.

Single-user file access: If two users ran the application at the same time on a network share, they could overwrite each other's data. File-based storage does not support concurrent access.

Admin read-only: The admin can view everything but cannot delete users, revoke company accounts, or remove job postings.

No application rejection: The system only supports 'Accepted' status. There is no way for a company to formally reject an applicant with a notification.

No logout from dialogs: Dialogs (View Applicants, My Applications) have no session timeout.

Future Improvements

- **Database Integration:** Replace .txt files with SQLite (embedded) or MySQL for concurrent multi-user access, proper querying, and data integrity.
- **Password Hashing:** Use SHA-256 or BCrypt to hash passwords before saving. Never store plain-text passwords in production software.
- **Admin Management Tools:** Allow admin to delete accounts, ban companies, remove inappropriate job postings, and view audit logs.
- **Rejection Notification:** Add a 'Reject' button in View Applicants that sets status to 'Rejected' and shows the employee a rejection message on next login.
- **Resume Upload:** Allow employees to attach a resume (PDF) to their application using JFileChooser and file copying.
- **Email Notifications:** Use JavaMail API to send email when an application is accepted or rejected.
- **Search by Category:** Add job category tags (Engineering, Design, IT) and allow filtering by category in addition to keyword search.
- **Company Rating System:** After completing a temp job, allow employees to rate the company, and allow companies to rate employees.